

Day 02 Python 期末复习笔记合集

按 day 目录合并生成

来源：C:\Users\50469\python-learning\期末复习笔记\day02

目录

1. python-strings.md
2. theory_01_fstring.md
3. theory_02_slice.md
4. theory_04_listcomp_lambda_class.md
5. theory_06_dict.md
6. theory_17_tuple_set.md
7. theory_21_list_deep.md
8. theory_24_string_methods.md

1. python-strings.md

Python 字符串 - Java 对照速查 (完整版)

来源: Day 1 学习记录 + WeChat 9个字符串文件 + 学校实验报告

总览：Python 字符串操作 7 大模块

1	1.	'...' / '..." / "......" + \' \' " \\'
2	2.	% -> .format() -> f-string
3	3.	find() / index() / in / replace()
4	4.	split() / join() / +
5	5.	upper() / lower() / capitalize() / title()
6	6.	strip() / lstrip() / rstrip()
7	7.	center() / ljust() / rjust()

Java <-> Python 速查表

维度	Java	Python	备注
类型声明	<code>String s = "hi";</code>	<code>s = "hi"</code>	动态类型
不可变性	[OK]	[OK]	完全一致
定义方式	仅双引号	单/双/三引号	Python 更多选择
转义字符	<code>\\"\'\\</code>	完全一致	
格式化	<code>String.format()</code>	<code>f-string</code> (首选) / <code>.format()/%</code>	Python 有三代方案
长度	<code>s.length()</code>	<code>len(s)</code>	[注意] <code>len</code> 是函数不是方法
重复 N 次	需循环	<code>"=" * 40</code>	Python 独有
子串判断	<code>s.contains("a")</code>	<code>"a" in s</code>	和 <code>list</code> 统一用 <code>in</code>
查找位置	<code>s.indexOf("t")</code>	<code>s.find("t")</code>	找不到都返回 -1
查找 (指定起点)	<code>s.indexOf("t", 5)</code>	<code>s.find("t", 5)</code>	一致
全部替换	<code>s.replaceAll("a","b")</code>	<code>s.replace("a","b")</code>	
限制次数替换	需手写循环	<code>s.replace("a","b", 2)</code>	Python 更方便
分割	<code>s.split(",")</code>	<code>s.split(",")</code>	完全一致
限制次数分割	<code>s.split(",", 2)</code>	<code>s.split(",", 2)</code>	一致
拼接	<code>String.join("-", list)</code>	<code>"-".join(list)</code>	[注意] 方向相反!
直接拼接	<code>"a" + "b"</code>	<code>"a" + "b"</code>	完全一致
全大写	<code>s.toUpperCase()</code>	<code>s.upper()</code>	


```

age = 27

# 1. 使用 % 格式化
print("name: %s, age: %d" % (name, age))

# 2. 使用 .format() 方法
print("name: {1}, age: {0}".format(age, name))

# 3. 使用 f-string (Python 3.6+)
print(f"name: {name}, age: {age}")

# 4. 使用 % 格式化 (带小数)
print("%.2f" % (19/22))  # 86.36%

```

第三代: f-string (Python 3.6+, 现代首选)

```

age = 20
ave_score = 88.8534
gender = '男'

print(f"age: {age}, gender: {gender}, ave_score: {ave_score:.2f}")
# -> age: 20, gender: 男, ave_score: 88.85

```

:.2f 的含义拆解:

- : - 格式说明开始
- .2 - 保留 2 位小数
- f - float 浮点数格式

三代对比

```

name = "小明"
age = 27
ave = 88.856

# 1. 使用 % 格式化
print("name: %s, age: %d, ave: %.2f" % (name, age, ave))

# 2. 使用 .format() 方法
print("name: {}, age: {}, ave: {:.2f}".format(name, age, ave))

# 3. 使用 f-string (Python 3.6+)
print(f"name: {name}, age: {age}, ave: {ave:.2f}")

```

日常用 f-string, 需要模板复用时用 .format(), 维护老代码时会看到 %。

三、查找与替换

find() - 查找子串位置

```

string = "Python"
string.find('t')  # 2 - 索引
string.find('t', 5)  # 6 - 从索引 5 开始
string.find('z')  # -1 - 没找到

```

= Java indexOf("t") / indexOf("t", 5), 找不到都返回 -1。

Python 也有 index() 方法, 但找不到会抛异常。日常用 find() 更安全。

in - 子串存在判断

```

if "python" in "Life is short. I use python":
    # ...

```

```
print("■■■■")
```

= Java `s.contains("python")`，但 Python 的 `in` 对 list 也适用，语法统一。

replace() - 替换

```
s = "All things Are difficult before they Are easy Are good"
```

```
s.replace("Are", "are")      # ■■■■ -> 3■Are■■are
s.replace("Are", "are", 2)    # ■■■■2■ -> ■■■Are■■
```

[注意] 字符串不可变！`s.replace(...)` 返回新字符串，必须 `s = s.replace(...)` 重新赋值。

四、分割与拼接

split() - 分割字符串 -> 返回列表

```
s = "The more efforts you make,the more fortune you get"
```

```
s.split()                    # ■■■■■■■■■■
# -> ['The', 'more', 'efforts', 'you', 'make,the', 'more', 'fortune', 'you', 'get']
```

```
s.split('m')                 # ■■■■ 'm' ■■■■m ■■■■■■■■
# -> ['The ', 'ore efforts you ', 'ake,the ', 'ore fortune you get']
```

```
s.split('e', 2)              # ■ 'e' ■■■■■■■■2■ -> ■■3■■■
# -> ['Th', ' mor', ' efforts you make,the more fortune you get']
```

= Java `s.split(" ")`，参数语义完全一致。

join() - 拼接列表 -> 返回字符串

```
symbol = '*'
world = 'Python'
symbol.join(world)           # 'P*y*t*h*o*n' - ■■■■■■■■ *
```

```
# ■■■■
"-".join(["2026", "05", "07"]) # "2026-05-07"
"".join(["a", "b", "c"])       # "abc"
```

[注意] join 方向记反是高频错误！分隔符调 `.join(■■)`，不是列表调。

- [OK] `"-".join(list)` <- 分隔符在前

- [X] `list.join("-")` <- 报错！

+ - 直接拼接

```
star = 'Py'
end = 'thon'
star + end    # 'Python'
```

五、大小写转换

```
old = 'hello world'
```

```
old.upper()      # 'HELLO WORLD'   - ■■■■= Java toUpperCase()■
old.lower()      # 'hello world'   - ■■■■= Java toLowerCase()■
old.capitalize() # 'Hello world'   - ■■■■■■
old.title()      # 'Hello World'   - ■■■■■■■■
```

方法	效果	Java 对比
<code>upper()</code>	HELLO WORLD	<code>toUpperCase()</code>
<code>lower()</code>	hello world	<code>toLowerCase()</code>
<code>capitalize()</code>	Hello world	需手写
<code>title()</code>	Hello World	需手写

`capitalize()` vs `title()` 的区别: `capitalize()` 只变第一个单词首字母, `title()` 每个单词首字母都变。

六、空白处理

```
old = ' Life is short,Use Python! '
```

```
old.strip()   # 'Life is short,Use Python!' - ■■■■■■= Java trim()■
old.lstrip()  # 'Life is short,Use Python! ' - ■■■■■■
old.rstrip()  # ' Life is short,Use Python!' - ■■■■■■
```

```
# ■■■■■■■■■■Java ■■■■■■■■■■
"...hello...".strip(".") # -> "hello"
```

七、对齐与填充

```
sentence = 'hello world'          # ■■ = 11
```

```
sentence.center(13, '-')  # '-hello world-' - ■■■■■13■- ■■
sentence.ljust(13, '*')    # 'hello world**' - ■■■■* ■■■■
sentence.rjust(13, '%')    # '%hello world' - ■■■■% ■■■■
```

参数含义: `s.xxx(width, fillchar)`

- **width** - 目标总长度 (如果 $\leq$ 原字符串长度, 不填充)
- **fillchar** - 填充字符 (必须单字符, 不写默认空格)

常见陷阱 TOP 5

#	错误写法	问题	正确写法
1	<code>s.replace("a","b")</code>	没赋值, 结果被丢弃	<code>s = s.replace("a","b")</code>
2	<code>list.join("-")</code>	join 方向反了	<code>"-".join(list)</code>
3	<code>s.len()</code>	len 是函数不是方法	<code>len(s)</code>
4	<code>s[0] = "A"</code>	字符串不可变	<code>s = "A" + s[1:]</code>
5	<code>str s = "hi"</code>	Python 不声明类型	<code>s = "hi"</code>

全部来源文件索引

文件	模块	知识点
字符串表示.py	定义与转义	单/双/三引号、 <code>\'</code> <code>\"</code> 转义

文件	模块	知识点
格式化字符串f-string.py	格式化	f-string + :.2f 精度控制
格式化字符串format方法.py	格式化	.format() 三种占位 + :.2%
格式化字符串%.py	格式化	%s %d %.2f
字符串的查找与替换.py	查找替换	find() + replace() count
字符串的分割与拼接.py	分割拼接	split() 三种 + join() ++
字符串大小写转换.py	大小写	upper/lower/capitalize/title
字符串对齐.py	对齐	center/ljust/rjust
删除字符串的指定字符.py	空白处理	strip/lstrip/rstrip

2. theory_01_fstring.md

- 01-fstring

theory_01: f-string - 把变量塞进字符串

读完约 3 分钟。

相关笔记：类型转换 - 字符串方法

你已经会的两种方式

```
name = "██"
age = 20

# 1
print("██", name, "██", age, "█")

# 2
print("██" + name + "██" + str(age) + "█")
```

f-string: 第三种方式

```
print(f"██{name}██{age}█")
```

规则就两条：

1. 引号前写 **f**
2. 变量名放 **{}** 里

Python 自动去 **{}** 里取值，自动把数字转成字符串。

{} 里可以是任意计算

```
a = 10
b = 3

print(f"{a} + {b} = {a + b}")          # 10 + 3 = 13
print(f"{a} ████ {a ** 2}")           # 10 ████ 100
```

浮点数精度问题

```
price = 29.9
count = 3
print(price * count)                   # 89.69999999999999 <- █ 89.7█

# 1
print(f"██{price * count:.1f}█")      # █89.7█
print(f"██{price * count:.2f}█")      # █89.70████2█
```

金融/价格相关永远加 **:.2f**。

为什么会有精度问题（IEEE 754）

计算机用二进制存小数。十进制里能精确表示的 **0.1**，在二进制里是 **0.0001100110011...** 无限循环，必然被截断。

```
0.1 + 0.2    # 0.30000000000000004 <- 0.3
```

所以 $29.9 * 3 = 89.699\dots$ 不是 Python 的 bug，所有语言都这样。

需要精确计算时用 Decimal

```
from decimal import Decimal
result = Decimal("29.9") * 3    # 89.7
```

注意：必须用字符串 "29.9" 传入 Decimal。用 `Decimal(29.9)` 还是会有精度问题--因为 29.9 传进去之前就已经被 Python 截断了。

进阶规则：f-string 两条核心规则

规则一：`{}` 里放的是表达式，不是语句

```
name = ""
print(f"{name.upper()}")    # [OK]
# print(f"{if name: ...}")  # [X]
```

规则二：`!` 控制转换，`:` 控制格式

```
x = 3.14159
print(f"{x!r}")    # !r -> repr() '3.14159'
print(f"{x:.2f}")  # :.2f -> 3.14
```

符号	作用	等价写法
<code>!r</code>	<code>repr()</code> 表示（调试用）	<code>f"{repr(x)}"</code>
<code>!s</code>	<code>str()</code> 表示（默认，可省略）	<code>f"{str(x)}"</code>
<code>!a</code>	<code>ascii()</code> 表示（转义非ASCII字符）	<code>f"{ascii(x)}"</code>
<code>:.2f</code>	格式说明（： 开头，控制精度/对齐/进制）	-

`!` 和 `:` 可以同时用： `f"{x!r:.10}"` - 先 `repr` 再截断到10字符。

常见坑

```
# 1 f
print("Hello, {name}!")    # Hello, {name}! --

# 2 f-string {} []
names = ["", ""]
print(f"{names[0]}")    # OK f-string {}
# print(f"{names[:]}")  # [' ', '']
```

引号搭配

```
print(f"{''}")    # OK {}
# print(f"{}")    #
```

enumerate：循环带编号

```
names = ["", "", "", ""]
```

```
for i, name in enumerate(names, 1): # 1 = 1
    print(f"{i}{name}")
# 1 2 3 4
```

实战：三种格式化方式（你微信收到的学习文件）

1. % 格式化 -> %format.py

```
name = '小明'
age = 27
ave = 88.856
address = '北京市海淀区'
print("%s" % name)
print("%6d\n%s\n%.2f" % (age, address, ave))
```

要点：%s=字符串,%d=整数,%f=浮点数,%.2f=保留2位小数,%6d=宽度6右对齐。

2. format() 方法 -> format.py

```
name = '小明'
age = 27
address = '北京市海淀区'

print("{} {}".format(name)) # 小明
print("{}\n{}".format(age, address)) # 27
print("{}\n{}".format(age, name)) # 27
print("{}\n{}".format(name1=name, age1=age)) # 小明

point, total = 19, 22
print("{}: {:.2%}".format(point/total)) # 86.36%
```

要点：{} 是占位符，支持顺序/编号/名称三种定位方式，:.2% 自动转百分比。

3. f-string -> f-string.py

```
age = 20
ave_score = 88.8534
gender = '男'
print(f"{age}, {gender}, {ave_score:.2f}")
```

要点：最简洁的写法，f"... " 里用 {} 直接取值。

三种方式对比

方式	语法	推荐度
% 格式化	"%s %d" % (val1, val2)	旧式，看得懂就行
format()	"{} {}".format(val1, val2)	中间代，了解即可
f-string	f"{val1} {val2}"	主力，推荐

3. theory_02_slice.md

- 02-slice

theory_02: 切片 - 从序列里取一段

读完约 4 分钟。

相关笔记：列表深入操作 - 字符串方法

语法

```
seq[start : stop : step]
```

三条铁律：

1. 包含 start，不包含 stop（取到 stop 的前一个位置）
2. 省略 start 默认从 0 开始
3. 省略 stop 默认到最后

字符串切片

```
s = "0123456789"

s[0:3]    # "012"    -- 从0,1,2取到3
s[3:6]    # "345"    -- 从3,4,5
s[:4]     # "0123"   -- 从start取到0
s[6:]     # "6789"   -- 从stop取到
s[:]      # ""       -- 从start取到stop
```

列表切片

```
nums = [10, 20, 30, 40, 50, 60, 70]

nums[0:3]    # [10, 20, 30]
nums[:3]     # [10, 20, 30]    -- 从start
nums[4:]     # [50, 60, 70]    -- 从stop
```

为什么 stop 不包含

```
# stop - start = 3
s[0:3]    # 3 - 0 = 3 取到"012"
s[2:5]    # 5 - 2 = 3 取到"234"

# 从start取到stop
s[0:3]    # "012"
s[3:6]    # "345"    <- 从stop = 6取到start
```

步长（第三个参数）

```
s = "0123456789"

s[::2]     # "02468"    -- 从0取到8步长为2
s[::3]     # "036"      -- 从0取到6步长为3
s[::-1]    # "987654321" -- 从-1取到-9步长为-1
```

负索引

```
s = "0 1 2 3 4 5 6 7 8 9"
s[-10]: -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
```

越界行为（重要）

```
s = "0000000000000000"

s[100]      # IndexError -- 越界
s[0:100]    # 000000000000 -> 00000000
```

规则：单元索引越界 = 报错。切片越界 = 自动截断，不报错。

步长方向陷阱

默认步长是 +1（从左往右走），所以 start 必须在 stop 的左边才能取到值。

```
s[-3:-1]    # "00" -- start 0 stop 2
s[-1:-3]    # ""   -- start 2 stop 0 -> 越界

# 0000000000000000
s[-1:-3:-1] # "00" -- 2-1=1
```

备考相关

- EXAM_PREP/day02/00_今日任务 - Day 2 字符串与组合数据类型

4. theory_04_listcomp_lambda_class.md

- 04-listcomp-class

Day 4: 列表推导式 + lambda + 类/继承

读完约 10 分钟（分两次读也行）。

一、列表推导式（Day2 补课，今天应该先学这个）

你之前学过了循环：

```
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
# squares = [1, 4, 9, 16, 25]
```

列表推导式是同一个逻辑的简写：

```
squares = [x ** 2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]
```

语法拆解：

```
[x for x in range(1, 6)]
  ↑           ↑           ↑
  x ** 2      for x in range(1,6)  ( )
```

带过滤的

```
# 
evens = []
for x in range(10):
    if x % 2 == 0:
        evens.append(x)

# 
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Java 对照：Stream API

```
// Java
List<Integer> squares = IntStream.range(1, 6)
    .map(x -> x * x)
    .boxed()
    .collect(Collectors.toList());
```

```
# Python 
squares = [x ** 2 for x in range(1, 6)]
```

推导式更简洁，但不要写太复杂--超过一层 for 就回去写普通循环。

二、lambda：匿名函数

什么时候用

```
# 
```

```
def square(x):
    return x ** 2

# lambda
lambda x: x ** 2
```

lambda 只在你需要一个「只用一次的小函数」时用。能写 def 就写 def，别硬用 lambda。

语法

```
lambda 参数: 表达式
```

只能写一个表达式（不能写多行、不能写 return、不能写 if 语句）。

配合 map/filter 使用

```
nums = [1, 2, 3, 4, 5]

# map
list(map(lambda x: x ** 2, nums))
# [1, 4, 9, 16, 25]

# filter
list(filter(lambda x: x % 2 == 0, nums))
# [2, 4]
```

但说实话：列表推导式比 map/filter 更 Pythonic：

```
# map
list(map(lambda x: x ** 2, nums))

# list comprehension
[x ** 2 for x in nums]
```

结论：了解 lambda 是什么就行，日常能用推导式就用推导式。

三、类/继承：你有 Java 基础，这节很快

Python 的类 vs Java 的类

```
// Java
public class Dog {
    private String name;

    public Dog(String name) {
        this.name = name;
    }

    public void bark() {
        System.out.println(this.name + " 汪汪");
    }
}

Dog d = new Dog("旺仔");
d.bark();

# Python
class Dog:
    def __init__(self, name): # 相当于 Java 的 Dog(String name)
        self.name = name

    def bark(self): # self 相当于 Java 的 this
        print(f"{self.name} 汪汪")

d = Dog("旺仔")
```

d.bark()

一一对应

Java	Python	说明
class Dog {}	class Dog:	定义类，Python 用冒号不用花括号
public Dog(...)	def __init__(self, ...)	构造函数
this.name	self.name	当前实例的成员变量
new Dog()	Dog()	创建实例，Python 不需要 new
void bark()	def bark(self):	方法，self 必须显式写在参数里

你唯一要适应的：self 必须显式写

```
class Counter:
    def __init__(self):
        self.count = 0      # ■■■■■■

    def add(self, n):
        self.count += n     # ■ self.count ■■

    def show(self):
        print(f"■■■■■{self.count}")

c = Counter()
c.add(5)
c.add(3)
c.show()    # ■■■■■8
```

Java 里 this 是可写可不写的（除非重名），Python 里 self 必须写。这是最大的习惯差异。

继承

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    # <- ■■■■■■ Java ■ extends
    def speak(self):
        # ■■■■■■
        return f"{self.name} ■■"

class Cat(Animal):
    def speak(self):
        return f"{self.name} ■■"

d = Dog("■■")
c = Cat("■■")
print(d.speak())  # ■■ ■■
print(c.speak())  # ■■ ■■
```

跟 Java 一样：继承 -> 重写方法 -> 多态。

当前级别许可

5. theory_06_dict.md

- 06-dict

06 - 字典 dict

相关笔记：列表深入操作 - 元组与集合

一句话

字典是 key->value 的映射表，类似"查电话号码"：给你人名（key），找到号码（value）。

```
phone = {"张三": "138xxxx", "李四": "139xxxx"}
print(phone["张三"]) # 138xxxx
```

1. 创建字典

```
# 1. 创建字典
d1 = {"name": "张三", "age": 18}

# 2. dict() 函数
d2 = dict(name="张三", age=18) # key 和 value

# 3. {} 字典
d3 = {}
d4 = dict()
```

2. 增删改查

```
d = {}

# 1. 访问字典元素
d["name"] = "张三" # {"name": "张三"}
d["age"] = 18 # {"name": "张三", "age": 18}
d["name"] = "李四" # {"name": "李四", "age": 18} <- 修改

# 2. 检查字典元素
print(d["name"]) # "张三" <- 成功
# print(d["gender"]) # KeyError! <- 失败

print(d.get("name")) # "张三" <- 成功
print(d.get("gender")) # None <- 失败
print(d.get("gender", "张三")) # "张三" <- 成功

# 3. 检查字典元素是否存在
print("name" in d) # True
print("gender" in d) # False

# 4. 删除字典元素
del d["age"] # {"name": "张三"}
# del d["gender"] # KeyError! <- 失败
```

3. 遍历字典

```
d = {"a": 1, "b": 2, "c": 3}

# 1. 遍历字典
for k in d:
    print(k, d[k]) # a 1 / b 2 / c 3
```

```
# ■■■ key-value
for k, v in d.items():
    print(k, v)          # a 1 / b 2 / c 3

# ■■■ value
for v in d.values():
    print(v)             # 1 / 2 / 3
```

4. 典型用法：分组（Group By）

这是 Hot100 最常用的模式：

```
groups = {}                # key=■■■■, value=■■

for word in ["eat", "tea", "tan"]:
    key = "aet"             # ■■■■■■■■
    if key not in groups:   # ■■■■■->■■■
        groups[key] = []
    groups[key].append(word) # ■■■■

print(groups)
# {"aet": ["eat", "tea"], "ant": ["tan"]}
```

5. 注意事项

场景	正确写法	错误写法
key 不存在时取值	d.get(key) 或 if k in d:	d[key] 直接取
保证 key 存在	d.setdefault(key, [])	手动 if 判断（也行）
删除不存在的 key	if k in d: del d[k]	del d[k] 直接删

Hot100 关联

遇到 dict 用法的题目：

- 第1题 两数之和 -> 存差值，O(1) 查找
- 第49题 字母异位词分组 -> 排序后分组
- 第128题 最长连续序列 -> 集合判断存在

备考相关

- EXAM_PREP/day02/00_今日任务 - Day 2 字符串与组合数据类型
- EXAM_PREP/day07/00_今日任务 - Day 7 学生管理系统综合题

6. theory_17_tuple_set.md

- 17-tuple-set

元组 (Tuple) + 集合 (Set)

前置：已掌握 list 和 dict

目标：搞懂元组和集合是什么、怎么用、和 list/dict 的区别

用时：约 10 分钟

相关笔记：列表深入操作 - 字典 dict

一、元组 (Tuple)：不可变的列表

基本操作

```
# 元组 () 列表 []
t = (1, 2, 3)
print(t[0])      # 1 - 列表索引
print(t[1:3])    # (2, 3) - 列表切片

# 遍历
for x in t:
    print(x)

# 长度
print(len(t))    # 3
```

元组 vs 列表

```
# 列表
lst = [1, 2, 3]
lst[0] = 99      # [OK] [99, 2, 3]

# 元组
t = (1, 2, 3)
t[0] = 99        # [X] TypeError: 'tuple' object does not support item assignment
t.append(4)      # [X] 列表 append
t.remove(1)      # [X] 列表 remove
```

元组能做的操作：索引、切片、遍历、len()、count()、index()

元组不能做的操作：增、删、改

count() - 统计某个值出现了几次

```
scores.count(80)  # 3 - 80 出现了 3 次
scores.count(90)  # 2
scores.count(100) # 0 - 100 出现了 0 次
```

index() - 查找某个值第一次出现在哪个位置

```
scores.index(90)  # 1 - 90 第一次出现在索引 1
scores.index(80)  # 0 - 80 第一次出现在索引 0
scores.index(100) # [X] ValueError: 100 不在列表中 -> 索引
```

为什么需要元组？

```
# 1. 
d = {}
d[(" ", " ")] = 92    # [OK] 
d[[" ", " "]] = 92    # [X] TypeError: unhashable type: 'list'

# 2. 
def min_max(nums):
    return min(nums), max(nums) # 

result = min_max([3, 1, 4, 1, 5])
print(result)          # (1, 5)
print(type(result))    # <class 'tuple'>

# 
low, high = min_max([3, 1, 4, 1, 5])
print(low, high)      # 1 5

# 3. " "
# RGB
COUNTRY_CAPITAL = (" ", " ") # 
```

[!NOTE]- 我的理解：可哈希 vs 不可哈希

可哈希 = 一辈子固定不变，值永远不改

Python 能给它们算出一个唯一固定编号（哈希值），字典靠这个编号找数据。

| 可哈希（不可变） | 不可哈希（可变） |

|---|---|

| 整数、字符串、布尔、元组 | 列表、字典、集合 |

| 一旦创建，不能改里面任何元素 | 随时可以增删改，值会变，哈希值就乱了，字典没法用它当键 |

元组的特殊语法

```
# --
t1 = (1)      # int tuple
t2 = (1,)     # 
print(type(t1)) # <class 'int'>
print(type(t2)) # <class 'tuple'>

# 
t = 1, 2, 3
print(t)      # (1, 2, 3)
print(type(t)) # <class 'tuple'>

# 
a, b, c = (1, 2, 3)
print(a, b, c) # 1 2 3

# 
x, y = 10, 20
x, y = y, x    # (x, y) = (y, x)
print(x, y)    # 20 10
```

一句话记

元组 = 只读列表。不能增删改，但能做字典键、函数多返回值、拆包。

二、集合（Set）：无序、不重复

基本操作

```
# {}
s = {1, 2, 3, 2, 1}
print(s)          # {1, 2, 3} -

#
s = set()          # [OK]
d = {}             # [X]

#
s = {1, 2, 3}
s.add(4)           # {1, 2, 3, 4}
s.add(2)           # {1, 2, 3, 4} -
s.remove(3)        # {1, 2, 4}
# s.remove(99)      # [X] KeyError -
s.discard(99)      # [OK]
```

集合运算

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)       # : {1, 2, 3, 4, 5, 6}  a.union(b)
print(a & b)       # : {3, 4}             a.intersection(b)
print(a - b)       # : {1, 2}             a.difference(b)
print(a ^ b)       # : {1, 2, 5, 6}       a.symmetric_difference(b)

#
print(1 in a)      # True
print(10 in a)     # False
print(a.isdisjoint({7, 8})) # True -
print({1, 2}.issubset(a))  # True -
```

集合的常见用途

```
# 1.
nums = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(nums))
print(unique)      # [1, 2, 3, 4]

# 2. in
names_set = {"", "", ""}
print("" in names_set) # True - 0(1)

# 3.
old_users = {"", "", ""}
new_users = {"", "", ""}

added = new_users - old_users    # {'', ''}
removed = old_users - new_users  # {'', ''}
```

集合推导式

```
nums = [1, 2, 2, 3, 3, 3]
s = {x**2 for x in nums}
print(s) # {1, 4, 9} -
```

一句话记

集合 = 无序的不重复元素集。| 并集、& 交集、- 差集。去重和成员检查最快。

备考相关

- EXAM_PREP/day02/00_今日任务 - Day 2 字符串与组合数据类型

7. theory_21_list_deep.md

- 21-list-deep

列表深入操作

前置：已掌握列表基本用法

目标：覆盖列表所有常用操作

用时：约 8 分钟

相关笔记：切片操作 - 元组与集合 - 字典 dict

一、增删改查

```
lst = [1, 2, 3]

# ■
lst.append(4)          # [1, 2, 3, 4]
lst.insert(1, 99)      # [1, 99, 2, 3, 4]
lst.extend([5, 6])    # [1, 99, 2, 3, 4, 5, 6]

# ■
lst.pop()              # 6
lst.pop(1)             # 99
lst.remove(3)          # lst = [1, 2, 4, 5]
# lst.remove(999)      # [X] ValueError - 
lst.clear()            # lst = []

# ■
lst = [1, 2, 3]
lst[1] = 99            # [1, 99, 3]

# ■
lst = [10, 20, 30, 20, 40]
print(lst.index(20))   # 1
print(lst.index(20, 2)) # 3
print(lst.count(20))   # 2
print(30 in lst)       # True
```

二、排序和反转

```
# sort()
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)           # [1, 1, 3, 4, 5]

nums.sort(reverse=True)
print(nums)           # [5, 4, 3, 1, 1]

# sorted()
nums = [3, 1, 4]
sorted_nums = sorted(nums)
print(sorted_nums)     # [1, 3, 4]
print(nums)           # [3, 1, 4] - 

# reverse()
nums.reverse()
print(nums)           # [4, 1, 3]

# ■ key
students = [(" ", 92), (" ", 88), (" ", 95)]
```


8. theory_24_string_methods.md

- 24-string-methods

字符串方法：查找 / 替换 / 拆分 / 合并

前置：已掌握字符串定义和切片
目标：掌握 Python 最常用的 12 个字符串方法
用时：约 12 分钟
相关笔记：切片操作 - 列表深入操作

概览：字符串方法分类速查

类别	方法	作用
查找	<code>find()</code> / <code>index()</code>	找子串位置
判断	<code>startswith()</code> / <code>endswith()</code>	判断开头/结尾
判断	<code>isalpha()</code> / <code>isdigit()</code> / <code>isalnum()</code>	判断字符类型
修改	<code>replace()</code>	替换
修改	<code>upper()</code> / <code>lower()</code> / <code>title()</code> / <code>capitalize()</code> / <code>swapcase()</code>	大小写转换
修改	<code>strip()</code> / <code>lstrip()</code> / <code>rstrip()</code>	去除两端 / 左端 / 右端空白
修改	<code>center()</code> / <code>ljust()</code> / <code>rjust()</code>	对齐与填充
拆分合并	<code>split()</code> / <code>join()</code>	拆分成列表 / 合并成字符串

一、查找类

`find()` vs `index()`

```
text = "Hello, welcome to Python world"

# find
print(text.find("welcome")) # 7
print(text.find("Java"))    # -1

# index
print(text.index("welcome")) # 7
print(text.index("Java"))    # [X] ValueError!
```

用 `find` 还是 `index`? -- 不确定是否存在时用 `find`，确定一定存在时用 `index`。

`startswith()` / `endswith()`

```
filename = "report_2024.pdf"

print(filename.startswith("report")) # True
```



```
# 使用 join() 方法将列表中的元素连接成字符串
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence)  # "Python is awesome"
```

重要: `join()` 的调用对象是分隔符字符串, 参数是列表:

```
",".join(["a", "b", "c"])  # "a,b,c"
" ".join(["a", "b", "c"])  # "a b c"
"".join(["a", "b", "c"])  # "abc"
```

综合示例: CSV 数据解析

```
# 解析 CSV 数据
raw_data = " 1, 25, 30 \n 1, 30, 30 "

# 清洗数据
cleaned = raw_data.strip()
lines = cleaned.split("\n")
for line in lines:
    fields = [f.strip() for f in line.split(",")]
    print(fields)  # ['1', '25', '30'] ['1', '30', '30']
```

四、其他实用方法

```
# count 方法
text = "apple banana apple cherry apple"
print(text.count("apple"))  # 3

# len 方法
print(len("Hello"))  # 5

# in 方法
print("apple" in text)  # True
print("orange" in text)  # False
```

`in` 比 `find()` 更直观, 判断是否存在时优先用 `in`。

实战案例 (来自你微信里的 Python 学习文件)

案例1: 字符串表示 (引号与转义) -> 01.py

```
print('单引号')
print("双引号")
print("""三引号""")

# 转义字符
print("let's learn Python")
print('let\'s learn Python')
print("How do you spell the word \"Python\"?")
```

要点: 单引号 `'`、双引号 `"`、三引号 `"""` 都可以定义字符串。引号嵌套时用 `\` 转义。

案例2: 大小写转换 -> 02.py

```
old_string = 'hello world'
print(old_string.upper())  # HELLO WORLD
print(old_string.lower())  # hello world
print(old_string.capitalize())  # Hello world
print(old_string.title())  # Hello World
```

案例3: 查找与替换 -> 000000000000.py

```
# find() 000000
string = "Pythonntn"
print(string.find('t'))      # 2 - 00000't'00000
print(string.find('t', 5))   # 6 - 000050000

# replace() 00
string = "All things Are difficult before they Are easy Are good"
print(string.replace("Are", "are"))      # 00000
print(string.replace("Are", "are", 2))   # 0000020
```

案例4: 分割与拼接 -> 000000000000.py

```
# split() 00
string = "The more efforts you make,the more fortune you get"
print(string.split())      # 00000000
print(string.split('m'))   # 0'm'00
print(string.split('e', 2)) # 0'e'00000020

# join() 00 + 00000
print(''.join('Python'))   # P*y*t*h*o*n
print('Py' + 'thon')       # Python - +0000
```

案例5: 字符串对齐 -> 000000.py

```
sentence = 'hello world'
print(sentence.center(13, '-')) # -hello world- 000-00
print(sentence.ljust(13, '*'))  # hello world** 0000*00
print(sentence.rjust(13, '%'))  # %%hello world 0000%00
```

案例6: 删除指定字符 -> 000000000000.py

```
old = ' Life is short,Use Python!'
print(f"strip:{old.strip()}00") # 000000
print(f"lstrip:{old.lstrip()}00") # 000000
print(f"rstrip:{old.rstrip()}00") # 000000
```

总结速记

```
0000 find000000000000 in
0000 strip00000 lower/upper
0000 split0000 join0000.join(00)0
00000 isdigit/isalpha/isalnum

00000
00000 -> 0/0/0000'\ \'
0000 -> upper() / lower() / capitalize() / title()
0000 -> find() / replace()
0000 -> split() / join() / +
00 -> center() / ljust() / rjust()
000 -> strip() / lstrip() / rstrip()
```

备考相关

- EXAM_PREP/day02/00_今日任务 - Day 2 字符串与组合数据类型